

FinVerify Optimization

SQL Analysis Task 3 of 3

Device-Specific Friction

Measures the average time (in seconds) users take to move from profile completion to KYC submission, broken down by operating system, for users who signed up in the last 14 days.

SECTION 01: SQL Query

```
WITH stage_times AS ( SELECT u.user_id, u.device_os, MIN(CASE WHEN uoe.stage_name =
'profile_completed' THEN uoe.event_timestamp END) AS profile_completed_time, MIN(CASE WHEN
uoe.stage_name = 'kyc_submitted' THEN uoe.event_timestamp END) AS kyc_submitted_time FROM
users u INNER JOIN user_onboarding_events uoe ON u.user_id = uoe.user_id WHERE u.signup_date
>= CURRENT_DATE - INTERVAL '14 DAY' GROUP BY u.user_id, u.device_os ) SELECT device_os, ROUND(
AVG( EXTRACT(EPOCH FROM ( kyc_submitted_time - profile_completed_time )) ), 2 ) AS
avg_time_seconds FROM stage_times WHERE profile_completed_time IS NOT NULL AND
kyc_submitted_time IS NOT NULL GROUP BY device_os ORDER BY avg_time_seconds DESC;
```

SECTION 02: Expected Output

Actual output values depend on the underlying dataset. The table below illustrates the expected output format.

device_os	avg_time_seconds
Android	48.75
iOS	35.20
Web	28.90

SECTION 03: Query Explanation

Step 1 — JOIN & Filter Recent Users: We join the users table with user_onboarding_events on user_id. The WHERE clause restricts to users whose signup_date falls within the last 14 days from today, ensuring we only measure friction for the current version of the UI — not historical behaviour.

Step 2 — Extracting Stage Timestamps: Inside the CTE, MIN(CASE WHEN stage_name = 'profile_completed' THEN event_timestamp END) picks the earliest timestamp when each user completed their profile. Similarly for kyc_submitted. Using MIN handles cases where a user may have triggered the same event more than once.

Step 3 — Time Difference in Seconds: In the outer query, we subtract the two timestamps and use EXTRACT(EPOCH FROM ...) to convert the PostgreSQL interval into a plain number of seconds. This is then averaged across all users with AVG(...).

Step 4 — NULL Filter & Grouping: The WHERE ... IS NOT NULL clause removes users who never completed one of the two stages — we can only calculate a time difference for users who completed both. Results are grouped by device_os and sorted from slowest to fastest to surface problem devices first.

SECTION 04: Business Impact

01 Confirm the Bug Hypothesis: If Android consistently shows a higher avg_time_seconds than iOS and Web, it provides statistical evidence that the selfie-capture UI has a performance or UX issue on Android.

02 Prioritise Platform Fixes: Engineering can be directed to fix the highest-friction OS first, maximising the number of users unblocked per sprint.

03 Set SLA Benchmarks: Establishing a baseline 'healthy' transition time (e.g. under 30 seconds) lets the team alert on regressions after each release.

04 Reduce Abandonment Rate: Long times between profile completion and KYC submission correlate with higher drop-offs; reducing friction directly improves the overall funnel conversion rate.